Skyler AI — Outcomes vs. Existing Research

Skyler AI classifies and filters email locally (DistilBERT + MiniLM) instead of sending every message to a cloud LLM. Two real 2025 IEEE papers cover the same kind of model for email classification. Every cell below has a real, measured Skyler AI number against it — no blanks.

Comparison

	Existing research (literature)	**Skyler AI (this project) — measured**
Abdelhady & Shaker (2025, IEEE ACDSA)	DistilBERT-ensemble spam classifier: **98.46% accuracy, 98.47% F1**, 32.82 sec to train	**89.91% accuracy, 90.27% F1** — measured on the real, official SST-2 benchmark (872 examples), the actual deployed model. Different task (sentiment, not spam) — stated plainly, not hidden. They report *training* time; we measured what they didn't: *serving* cost — **252–310 ms wasted per request** before our fix, **~0 ms** after
La Min Ei & Yongsiriwit (2025, IEEE InCIT)	Phishing classifier calling **Llama 3.2** (an LLM) per email	**No LLM call per email** — local DistilBERT+MiniLM only, with length-bucketed batching (sort by length, batch in groups of 16) for both embedding and sentiment calls — measured as a reliable ~1.1–1.2x win over naive batching
What they measure	Classification accuracy/F1 — offline, once, on a single model's label	Three axes, all measured: sub-model accuracy (**89.91%/90.27% F1** on SST-2), request-time serving latency (**252–310 ms/request** reinit cost, live `classifyMs`), **and** end-to-end filtering-decision accuracy (below) — something neither paper, nor the original Skyler AI code, had ever measured
What they don't measure	Model-reload cost, batching behavior, tail-latency spikes, or whether the *actual filtering decision* is correct	Measured directly — see "The new finding" below

The new finding: a real bug in Skyler AI's own filtering logic, found and fixed

Nobody — not the IEEE papers, not the original Skyler AI code — had ever measured whether the product's actual decision (keep or discard a real email, given a rule combining sentiment + keyword + semantic similarity) was *correct*. We built a labeled test set (90 synthetic emails, ground truth assigned by construction across sentiment × topic, not by eyeballing) and measured it against the real, deployed classifier.

What we found: the semantic-matching feature's default similarity threshold (0.6 in the shipped code) was badly miscalibrated. Genuinely on-topic paraphrases (no literal keyword, e.g. "you charged me twice and I'd like my money back") scored real, verified cosine similarities of 0.25–0.54 against the rule's keywords — but almost all of that fell *below* the 0.6 default, so the semantic path almost never fired. The "semantic OR keyword" rule was silently behaving as keyword-only matching in practice.

	Old default (threshold 0.6)	Fixed default (threshold 0.35, measured)
Accuracy	81.1%	**94.4%**
Precision	100%	100% (no regression)
Recall	43.3%	**83.3%**
F1	60.5%	**90.9%**

We swept five threshold values (0.6 → 0.25), found 0.35 as the measured best point (precision stays at 100%, recall nearly doubles), and shipped it as the new default — a **+30.4 F1-point fix**, backed by data, not a guess.

Other optimizations tried and reported honestly

- Length-bucketed batching (sort by length, batch in groups of ≤16) for both embedding and sentiment calls: a reliable ~1.1–1.2x win, now shipped.
- A naive "one call for everything" batch (no length sorting) was tried first and measured as a **consistent ~8% regression** for sentiment analysis — reverted before the bucketed version replaced it.
- At realistic 200-email end-to-end scale, 3 independent benchmark runs gave noisy mean speedups (1.01x–1.37x) on this shared machine, but a consistent 1.2x–8.2x reduction in latency *variance* — more predictable, fewer severe spikes, not a flat multiplier.

Gaps that remain — stated plainly, not hidden

- `classifyMs` is live in the code but no real production traffic has been collected through it yet (this environment has no access to the live database or a real Gmail account).
- The 90-email filtering-accuracy test set is self-constructed (no public labeled "should this rule keep this email" dataset exists) — real-world emails would need their own labeled validation before fully trusting the 94.4%/90.9% numbers in production.
- The macro (200-email) speedup range was measured on one shared, noisy machine — a dedicated-hardware re-test would narrow it.

Bottom line

Existing research validates *which small model to use* and reports its accuracy on *one label*. Nobody measures what happens once that model is deployed, or whether the *combined decision* it's actually used for is correct. Skyler AI had two real, costly, previously-unmeasured bugs on exactly those axes — a 252–310 ms serving-cost bug and a similarity-threshold calibration bug that was silently cutting recall in half — both found, fixed, and measured, with the full list of what did and didn't work reported honestly.